

Nama : Nico Arie Angga Barus
NPM : G1D021065
Mata kuliah : Optimisasi
Tugas : Merumuskan masalah

```
julia> using Printf #packges yang digunakan adalah packges Printf
```

```
julia> const max_speed = 100 #kecepatan maksimum yang digunakan adalah 100
```

```
julia> const min_speed = 0 #kecepatan minimum yang digunakan adalah 0
```

```
julia> function read_sensors() #  
    return [0, 1, 1, 0, 0]  
end
```

read_sensors (generic function with 1 method)

#Fungsi ini membaca sensor pembacaan. Dalam contoh ini, fungsi mengembalikan array [0, 1, 1, 0, 0], di mana 1 menunjukkan bahwa garis terdeteksi dan 0 menunjukkan tidak terdeteksi. Ini bisa dianggap sebagai data dari sensor garis yang sering digunakan dalam robotika.

```
julia> function calculate_error(sensor_values)  
    return -2 * sensor_values[1] + -1 * sensor_values[2] + 0 * sensor_values[3] + 1 *  
    sensor_values[4] + 2 * sensor_values[5]  
end
```

calculate_error (generic function with 1 method)

#Fungsi ini menghitung nilai kesalahan (error) berdasarkan pembacaan sensor. Dalam hal ini, setiap sensor memiliki bobot yang berbeda. Sensor yang mendeteksi garis di sebelah kiri akan memberikan nilai negatif, sedangkan sensor di sebelah kanan memberikan nilai positif. Ini membantu menentukan arah yang perlu diambil oleh robot untuk tetap berada di garis.

```
julia> function pid_control(error)
```

```
    Kp = 1.0 #Gain Proportional
```

```
    Ki = 0.0 # Gain Integral
```

```
    Kd = 0.0 #Gain Derivative
```

```
    adjustment = Kp * error
```

```
    left_speed = clamp(max_speed - adjustment, min_speed, max_speed)
```

```
    right_speed = clamp(max_speed + adjustment, min_speed, max_speed)
```

```
    return left_speed, right_speed
```

```
end
```

```
pid_control (generic function with 1 method)
```

#Fungsi ini mengimplementasikan kontrol PID (Proportional-Integral-Derivative), meskipun hanya proporsional (Kp) yang digunakan. **Adjustment** dihitung berdasarkan nilai error dan konstanta Kp. Kecepatan motor kiri (**left_speed**) dan kanan (**right_speed**) dihitung dengan mengurangi dan menambahkan **adjustment** dari **max_speed**, kemudian dibatasi agar tidak melebihi batas minimum dan maksimum.

```
julia> function control_motors(left_speed, right_speed)
```

```
    @printf("Left Motor Speed: %.2f, Right Motor Speed: %.2f\n", left_speed, right_speed)
```

```
end
```

```
control_motors (generic function with 1 method)
```

#Fungsi ini bertugas untuk menampilkan kecepatan motor kiri dan kanan.

Menggunakan **@printf**, mencetak kecepatan dengan format dua angka desimal.

```
julia> output_displayed = false  
false
```

```
julia> while true  
    sensor_values = read_sensors()  
    error = calculate_error(sensor_values)  
    left_motor_speed, right_motor_speed = pid_control(error)  
  
    if !output_displayed  
        control_motors(left_motor_speed, right_motor_speed)  
        output_displayed = true  
    end  
  
    sleep(0.1)  
end
```

#Dari sini di miliki loop utama yang berjalan selamanya (**while true**). Dalam setiap iterasi:

- Nilai sensor terbaca.
- Kes
- Kecepatan motor dihitung dengan kontrol PID.
- Jika output belum ditampilkan (**output_displayed** adalah **false**), **output_displayed** diubah menjadi **true**.

Loop ini juga memiliki **sleep(0.1)** yang membuat program berhenti sejenak selama 0,1 detik sebelum iterasi berikutnya, memberikan waktu bagi sistem untuk tidak terlalu cepat dalam membaca dan menampilkan output.

Left Motor Speed: 100.00, Right Motor Speed: 99.00

#Ini menunjukkan bahwa kecepatan motor kiri adalah 100 dan motor kanan adalah 99, yang mencerminkan penyesuaian berdasarkan nilai error yang dihitung dari sensor.

```
Julia 1.10.5

julia> using Printf

julia> const max_speed = 100
100

julia> const min_speed = 0
0

julia> function read_sensors()
    #Mengembalikan nilai sensor sebagai array
    #1 berarti garis terdeteksi, 0 berarti tidak terdeteksi
    #Contoh nilai sensor
    return [0, 1, 1, 0, 0] #contoh nilai sensor
end
read_sensors (generic function with 1 method)

julia> function calculate_error(sensor_values)
    return -2 * sensor_values[1] + -1 * sensor_values[2] + 0 * sensor_values[3] + 1 * sensor_values[4] + 2 * sensor_values[5]
end
calculate_error (generic function with 1 method)

julia> function pid_control(error)
    Kp = 1.0
    Ki = 0.0
    Kd = 0.0

    adjustment = Kp * error

    left_speed = clamp(max_speed - adjustment, min_speed, max_speed)
    right_speed = clamp(max_speed + adjustment, min_speed, max_speed)

    return left_speed, right_speed
end
pid_control (generic function with 1 method)

julia> function control_motors(left_speed, right_speed)
    @printf("Left Motor Speed: %.2f, Right Motor Speed: %.2f\n", left_speed, right_speed)
end
control_motors (generic function with 1 method)

julia> output_displayed = false
false

julia> while true
    sensor_values = read_sensors()
    error = calculate_error(sensor_values)
    left_motor_speed, right_motor_speed = pid_control(error)

    if !output_displayed
        control_motors(left_motor_speed, right_motor_speed)
        output_displayed = true
    end

    sleep(0.1)
end
Left Motor Speed: 100.00, Right Motor Speed: 99.00
```